

SMARTMATCH

AI-POWERED INTERNSHIP RECOMMENDATION SYSTEM

FINAL PROJECT REPORT

Implementation, Validation and Testing Report

Name : Shreyas R

Studying : B.Tech

College: Sapthagiri NPS University

Prepared from the working SmartMatch project files and the completed 18-test validation cycle.

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope
6. Existing System
7. Proposed System
8. System Requirements and Technology Stack
9. System Architecture
10. Dataset and Data Generation
11. Frontend Implementation
12. Backend and API Implementation
13. Matching Engine and Skill Normalization
14. Scoring Algorithm
15. Recommendation Ranking and Explainability
16. Validation and Error Handling
17. Testing Methodology and Results
18. Defects Identified and Corrective Actions
19. Limitations
20. Future Enhancements
21. Conclusion
21. Testing Summary
22. References and Source Basis

1. Abstract

SmartMatch is a web-based internship recommendation system designed to reduce the effort involved in manually comparing a candidate profile against a collection of internship opportunities. The implemented system accepts a candidate's name, skills, location, experience, and education through a simple browser interface. A FastAPI backend receives the request, loads the opportunity dataset, and evaluates the candidate against every available opportunity. The matching engine normalizes skills, handles selected skill aliases, calculates skill similarity, and combines skill, location, experience, and education scores into a weighted final score. Recommendations are sorted in descending order and the top ten are returned to the frontend. The interface explains each recommendation through matched skills, missing skills, required skills, and a score visualization. The completed validation cycle covered 18 functional and robustness tests, including API validation, edge cases, normalization, ranking, and an end-to-end frontend test. All 18 tests passed after identified defects were corrected and retested.

2. Introduction

Internship selection is often performed by scanning many job descriptions and manually comparing them with a student's skills, qualification, experience, and preferred location. That process becomes increasingly repetitive when the number of opportunities grows. SmartMatch addresses this practical problem with a lightweight recommendation pipeline that can evaluate one candidate against the full opportunity dataset and present a ranked shortlist. The project is deliberately designed as a modular application: the browser handles user interaction, FastAPI provides the service layer, and a separate matching engine performs the scoring work. This separation makes the implementation easier to test and extend. The system also focuses on interpretability. Instead of returning only a numerical score, it identifies which required skills are already present and which skills are missing. This gives the candidate useful information for deciding where to apply and what to improve. The current implementation is a working prototype suitable for demonstration and further development.

3. Problem Statement

The central problem addressed by SmartMatch is the difficulty of consistently ranking internship opportunities for an individual candidate. A manual process can overlook relevant opportunities, apply different judgement from one candidate to another, and make it difficult to explain why one opening was preferred over another. The implemented system treats internship recommendation as a structured comparison problem. Each opportunity contains a company, role, required skills, location, minimum experience, and education requirement. A candidate provides the corresponding profile attributes. The engine then produces a compatibility score using four dimensions rather than relying on one field alone. Skills receive the largest weight because they are the most direct indication of technical fit. Location, experience, and education contribute additional context. The result is a ranked list that can be reproduced for the same inputs. The project therefore focuses on a practical recommendation workflow rather than claiming a fully trained production machine-learning model.

4. Objectives

The primary objective of SmartMatch is to build a usable internship recommendation workflow from candidate input to ranked opportunities. The implementation has several supporting objectives. First, it should accept structured candidate information through a simple frontend. Second, the backend should validate the request before processing it. Third, the matching engine should normalize skill text so that capitalization, spacing, delimiters, and selected aliases do not unnecessarily change the result. Fourth, the system should calculate a transparent weighted score using skills, location, experience, and education. Fifth, the API should return enough information for the frontend to explain the recommendation rather than showing a score alone. Sixth, recommendations should be sorted from highest to lowest score and limited to the top ten for a clean interface. Finally, the system should be robust enough to handle malformed requests and unusual inputs. These objectives were validated through the 18-test campaign completed during implementation.

5. Scope

The current scope covers a browser-based candidate form, a FastAPI service, a CSV-based opportunity dataset, a Python matching engine, and a ranked recommendation display. The system can compare an entered candidate against all 30 opportunities currently stored in the dataset and return the ten highest-scoring results. The matching engine also supports a batch mode that compares all 50 stored intern profiles with all 30 opportunities, producing 1,500 pairwise match records in the generated matches file. The scope is intentionally limited to structured profile fields. It does not currently extract skills automatically from resumes, authenticate users, store profiles in a production database, or connect to live internship portals. It also does not implement the one-to-one Hungarian assignment described in the separate reference PDF found in the uploaded folder; the working SmartMatch application instead performs independent candidate-to-opportunity scoring and Top-10 ranking. Keeping this distinction explicit is important for an accurate academic report.

6. Existing System

Traditional internship shortlisting is frequently based on manual browsing, keyword searching, spreadsheets, or simple filters such as degree and experience. These approaches can be useful for a small number of opportunities, but they become less convenient as the candidate or opportunity pool grows. Manual review also makes it harder to maintain a consistent ranking rule. A keyword-only approach has another weakness: formatting differences such as uppercase text, extra spaces, or common aliases can produce inconsistent results. SmartMatch improves this workflow by applying the same scoring procedure to every opportunity for a given candidate. The system also exposes missing skills, which is more informative than a simple yes/no filter. Another practical advantage is reproducibility: the same profile is processed using the same scoring rules and dataset. The existing approach is therefore best viewed as a human-intensive baseline, while SmartMatch provides a repeatable computational shortlist that can support, rather than replace, human decision-making.

7. Proposed System

The proposed SmartMatch system consists of three main application layers. The frontend is a static HTML, CSS, and JavaScript page where a candidate enters profile information and requests recommendations. The backend is implemented with FastAPI and exposes endpoints for the application root, opportunity retrieval, and matching. The matching engine is a separate Python module that performs skill normalization, similarity calculation, matched-skill extraction, missing-skill extraction, and the weighted score calculation. When a request reaches the /matches endpoint, the backend loads the opportunities CSV, constructs an internal candidate dictionary, evaluates every opportunity, sorts the resulting records in descending order, and returns the top ten. The frontend then renders each result as a recommendation card. The card contains company, role, location, required skills, matched skills, missing skills, an explanation, and a score bar. This design keeps user interaction, API processing, and recommendation logic separated, which makes debugging and future replacement of components easier.

8. System Requirements and Technology Stack

The working prototype uses a lightweight technology stack chosen for simplicity and clear separation of responsibilities. Python is used for backend processing and the recommendation engine. FastAPI provides the web API, while Pydantic supplies request-model validation. The frontend is a standalone HTML page using CSS for layout and JavaScript for collecting form values, sending the POST request, and rendering the returned results. CSV files provide the current data source, which avoids the additional setup required by a database during development. The matching engine uses Python's standard csv, os, and re modules for data and text handling. The final uploaded project also contains separate requirements files for the machine-learning engine and its backend environment. During development, the application was run locally with Uvicorn on port 8000 and the frontend through a local web server on port 5500. CORS was explicitly configured for the two local frontend origins so that browser requests could reach the API.

9. System Architecture

SmartMatch follows a simple request-response architecture. At the user interface layer, the candidate enters five fields: name, skills, location, experience, and education. JavaScript validates that the fields are filled and converts the experience field to a number before constructing the request object. The browser sends a JSON POST request to the FastAPI /matches endpoint. FastAPI validates the request using the InternRequest Pydantic model. The backend then loads the opportunity records and calls the matching engine for each opportunity. The engine independently calculates skill, location, experience, and education components and returns a weighted final score. The backend also asks the engine for matched and missing skills, assembles the response objects, sorts them by score, and keeps the first ten. The response travels back to the browser as JSON. The frontend creates result cards dynamically and highlights the first result as the top match. This architecture makes the flow easy to follow and provides clear boundaries for testing.

10. Dataset and Data Generation

The project uses CSV datasets generated locally for controlled testing. The intern dataset contains 50 records and six fields: id, name, skills, location, experience, and education. The opportunity dataset contains 30 records and seven fields: id, company, role, skills, location, min_experience, and education. The generator scripts use a fixed random seed of 42 and randomly select skills, locations, education values, and experience values from predefined pools. This creates repeatable synthetic data while retaining enough variation for testing. The working matching engine also generates a batch matches file containing 1,500 records, corresponding to every combination of 50 interns and 30 opportunities. The current dataset is synthetic and therefore should not be interpreted as a real labor-market dataset. Its purpose is to exercise the recommendation logic under controlled conditions. For a production system, the same schema could be backed by a database or live opportunity source, and candidate skills could be extracted automatically from resumes.

11. Frontend Implementation

The frontend is implemented as a single HTML page with embedded CSS and JavaScript. The interface uses a centered card layout and provides fields for the candidate's name, skills, location, experience, and education. When the user clicks the matching button, the page clears previous results, displays a loading message, validates that all fields are populated, and sends the request to the backend. The JavaScript converts comma-separated skills into an array before serializing the request as JSON. After receiving the response, it displays the results card and creates one recommendation element for each returned match. The first result receives a top-match class and badge. Each card displays the role and company information, location, matched skills, missing skills, required skills, and score. The interface also generates a plain-language explanation from the score range and uses a progress bar whose width reflects the score. An escapeHTML helper is included before inserting dynamic values into the page, reducing the risk of raw HTML being interpreted as markup.

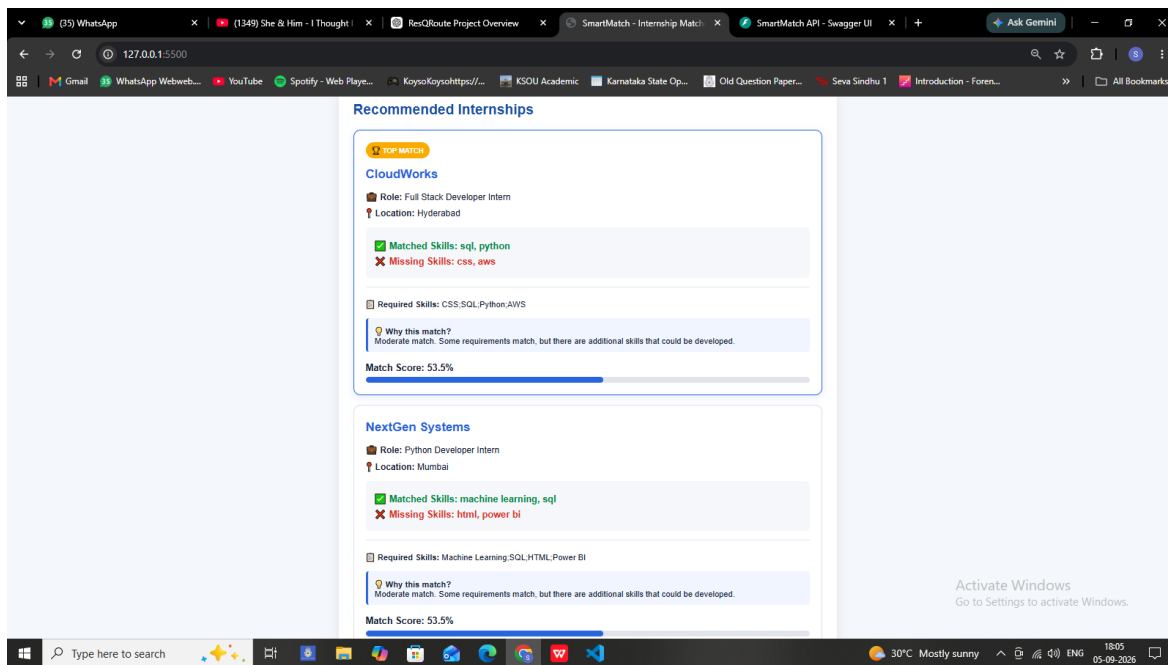


Figure 1. Test 16 — location and education variation results.

12. Backend and API Implementation

The backend is a FastAPI application defined in `backend/main.py`. It establishes the project base directory, adds the `ml-engine` folder to Python's import path, and imports the matching functions. The root endpoint confirms that the service is running. The `/opportunities` endpoint loads the opportunity CSV and returns the total count and records. The main `/matches` endpoint accepts an `InternRequest` model. The request model requires name, skills, location, and education and requires experience to be an integer greater than or equal to zero through Pydantic's `Field` constraint. For each opportunity, the endpoint calculates the overall score, extracts matched skills, extracts missing skills, and adds the required skills from the opportunity record. Results are sorted in reverse score order and the first ten are returned. CORS allows the local frontend origins used during development. This API design was directly exercised through Swagger and the browser, and malformed requests were used to verify that validation errors return HTTP 422 responses.

13. Matching Engine and Skill Normalization

The matching engine is implemented separately from the API in `ml-engine/matching_engine.py`. Skill input is split using both commas and semicolons, stripped of surrounding whitespace, converted to lowercase, and deduplicated. A controlled alias dictionary maps selected alternatives such as ML to machine learning, ReactJS to react, NodeJS to node.js, Postgres to postgresql, and Py to python. The final `skills_match` function uses exact normalized equality, meaning a skill is considered matched only when the normalized forms are equal. This design was introduced after testing exposed two important false positives: Java had been matching JavaScript, and SQL had been matching PostgreSQL. Those errors were caused by broad substring-style matching and overly aggressive aliasing. The corrected exact-match approach prevents these accidental relationships while preserving deliberate aliases. The test suite also confirmed that uppercase input and extra spaces do not change the underlying normalized skill representation.

14. Scoring Algorithm

The final SmartMatch score is a weighted combination of four independently calculated components. Skill similarity carries 65 percent of the final score. The engine counts how many required opportunity skills are present in the candidate's normalized skill set and divides that number by the number of required skills. Location contributes 15 percent: an exact location match receives 100, while a non-matching location receives 40. Experience contributes 10 percent. If an opportunity requires no experience or the candidate meets the minimum, the experience component is 100; otherwise a proportional value is calculated. Education contributes the final 10 percent. An exact education match receives 100, related engineering degrees receive 90, and other non-exact combinations receive 50. The weighted formula is therefore: $\text{final} = \text{skill} \times 0.65 + \text{location} \times 0.15 + \text{experience} \times 0.10 + \text{education} \times 0.10$. The engine rounds component values and the final score to two decimal places before returning them.

15. Recommendation Ranking and Explainability

After calculating scores for every opportunity, the backend sorts the complete result list in descending order of `match_score` and returns only the first ten records. This means the ranking is produced from the same scoring function for every opportunity rather than from a manually selected shortlist. The frontend adds a visual top-match badge to the first returned record, while the remaining cards preserve the same information structure. Explainability is handled at two levels. First, the response explicitly lists matched skills and missing skills. Second, the frontend translates the score range into a short explanation such as excellent, good, moderate, or lower match. The matching engine also contains a more detailed explanation generator used when creating the batch matches CSV. This makes the system more useful to a candidate because a score can be interpreted in context. Test 17 specifically checked the descending ranking order and confirmed that all ten displayed scores were correctly ordered.

16. Validation and Error Handling

Validation is implemented at both the browser and API levels. The frontend checks that all five fields have values before making a request and displays a message when information is missing. The backend provides the stronger contract through Pydantic. The experience field is an integer constrained to values greater than or equal to zero, which means a negative value is rejected with a 422 validation response. A non-numeric string such as two is also rejected because it cannot be parsed as an integer. Missing required fields similarly produce a 422 response that identifies the missing location in the request body. The frontend checks response.ok and attempts to read the server's detail message before falling back to a generic connection message. This layered approach was important during testing because it allowed us to distinguish user-input problems from server connectivity problems. The final test campaign confirmed successful handling of missing fields, invalid types, negative experience, empty skills, zero experience, and unknown skills.

17. Testing Methodology and Results

Testing was conducted incrementally while the application was being developed. The first group of tests focused on normal profiles and recommendation behavior. Additional tests then targeted known weaknesses in skill matching, such as Java versus JavaScript and SQL versus PostgreSQL. API-level tests were used to verify HTTP status codes and request validation. Edge-case tests checked empty skills, zero experience, negative experience, unknown skills, case differences, and extra whitespace. Later tests examined location and education variation, ranking order, and the complete frontend-to-backend flow. The testing process was not limited to checking whether a page loaded; it inspected returned scores, matched skills, missing skills, and ordering. When a defect was found, the implementation was changed and the relevant scenario was retested. The final campaign contains 18 completed tests, all marked PASS after corrections. This provides evidence that the current prototype is functional across its primary user journey and its most important input-handling and matching behaviors.

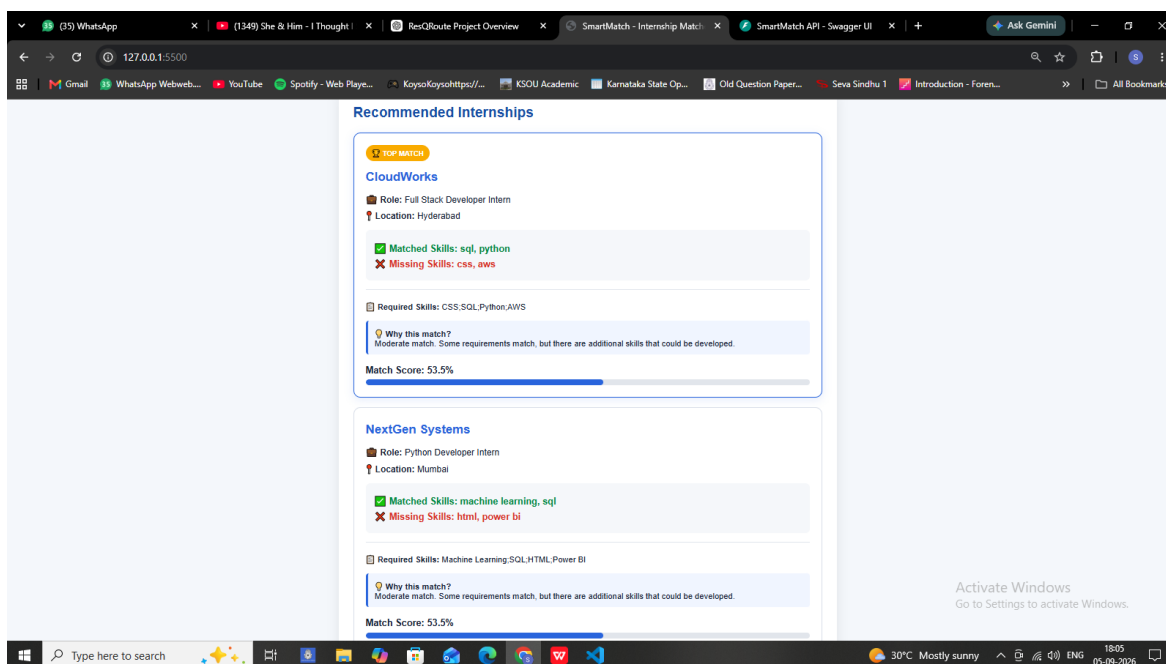


Figure 1. Test 16 — location and education variation results.

18. Defects Identified and Corrective Actions

The development process produced several useful defects. The first was a false skill relationship between Java and JavaScript. The original matching approach allowed substring relationships, so Java could be treated as part of JavaScript. The second was a SQL/PostgreSQL false match caused by an overly broad alias strategy. Both issues were addressed by using exact normalized equality and retaining only deliberate aliases. A separate response-format issue caused the frontend to show required skills as unavailable because the backend response did not include the required_skills field. That field was added and the frontend then displayed the opportunity requirements correctly. Another validation weakness allowed negative experience to reach the scoring engine. The request model was strengthened with `Field(ge=0)`, and the negative-value test was repeated successfully. Finally, the development environment exposed a Uvicorn reload limitation: changes in the separate ml-engine directory were not automatically watched by the backend reload process, so the server had to be restarted manually after matching-engine changes. These fixes were incorporated before the final validation.

19. Limitations

The current system is a functional prototype and has several boundaries that should be stated clearly. The opportunity and intern data are synthetic CSV records, so the recommendations are demonstrations of the scoring logic rather than predictions about real internships. Candidate information is entered manually; the system does not yet parse resumes or extract skills from unstructured documents. The matching model is also rule-based in its final score: although the project uses a separate matching-engine module, the working endpoint calculates deterministic weighted components rather than learning weights from historical hiring outcomes. Location is represented with a simple exact-match versus fallback score and does not account for commuting distance or remote flexibility. Education is similarly simplified. The current API has no authentication, database layer, user account system, or live job-feed integration. These limitations do not prevent the prototype from demonstrating the intended recommendation workflow, but they should be acknowledged before presenting the system as a production deployment.

20. Future Enhancements

The next version of SmartMatch could extend the existing modular design in several directions. Resume parsing could automatically extract skills, education, and experience from uploaded documents, reducing manual entry. A database could replace CSV storage and support frequent opportunity updates, user accounts, and historical recommendations. The scoring weights could be learned from real application or placement outcomes rather than fixed manually. Semantic embeddings could also be introduced so that related technical concepts can be recognized without unsafe substring rules. Location matching could become distance-aware and could include remote or hybrid preferences. The interface could provide filters for role, company, score threshold, or missing skills. A feedback mechanism could allow candidates to mark useful or unsuitable recommendations, creating data for future ranking improvements. Explainability could be expanded into a component-level score breakdown visible to users. Finally, automated unit and integration tests could be added to the repository so that future code changes can be validated automatically before deployment.

21. Conclusion

SmartMatch successfully demonstrates a complete internship recommendation workflow from candidate input to ranked and explainable results. The application combines a browser frontend, a FastAPI service, a dedicated matching engine, and structured CSV datasets. The core scoring model gives the greatest importance to skills while also incorporating location, experience, and education. The engine normalizes user-entered skills and supports selected aliases, while exact normalized matching prevents the false positives discovered during development. The API validates input and returns structured recommendation data, and the frontend presents that data in a form that is easy to understand. Most importantly, the project was validated through 18 tests covering normal usage, API behavior, edge cases, matching correctness, ranking, and end-to-end operation. All 18 tests passed after identified defects were corrected and retested. The result is a credible working prototype that can serve as a foundation for future enhancements such as resume parsing, live opportunity data, learned ranking models, richer location preferences, and production-grade persistence.

21. Detailed Test Cases

The following 18 test cases document the validation work performed during implementation. Each case records the purpose, inputs, observed behavior, and final status. Tests that initially exposed defects are described honestly, including the corrective action and retest.

Test 01 — Rahul: Normal Multi-Skill Profile

The first test established a normal baseline for the recommendation workflow. The candidate profile used Python, SQL, and Machine Learning, with Bangalore as the location, two years of experience, and B.Tech as the education. The purpose was to verify that a realistic multi-skill candidate produces multiple recommendations with meaningful scores rather than an empty or failed response. The results showed NextGen Systems' Python Developer Intern at 58.5 percent with machine learning and SQL matched, followed by CloudWorks' Full Stack Developer Intern at 57.5 percent with SQL and Python matched. Missing skills were also displayed. The result demonstrated that the engine considers multiple profile attributes and that the frontend can render a ranked shortlist. It became the baseline against which later normalization and matching corrections were compared. The test passed because the application accepted the profile, returned recommendations, calculated scores, and exposed matched and missing skills without an error.

FINAL STATUS: PASS

Test 02 — Arjun: Frontend Profile

The second test focused on a frontend-oriented profile using HTML, CSS, JavaScript, and React, with Bangalore, one year of experience, and B.Tech. This test was particularly useful because the skill list contained JavaScript, which exposed a weakness in the original substring-based matching logic. During the initial run, Java could be treated as a match for JavaScript. That behavior was not acceptable because Java and JavaScript are distinct technologies. The matching function was therefore corrected so that skills are compared using exact normalized equality, with explicit aliases handling only deliberate variations. The profile was then retested. The corrected system returned a WebWorks AI Engineering Intern recommendation with React, CSS, and HTML correctly identified as matched, while Power BI and MongoDB were shown as missing. The final result reached 73 percent. The test therefore passed after a genuine defect was identified, corrected, and verified again using the same scenario.

FINAL STATUS: PASS

Test 03 — Priya: SQL/PostgreSQL Validation

This test was designed specifically to verify that SQL should not automatically match PostgreSQL. The candidate used SQL, Excel, Data Analysis, and Python, with Hyderabad, one year of experience, and B.Tech. The initial implementation produced an incorrect result in which SQL was treated as equivalent to PostgreSQL, inflating the recommendation. Investigation found that the alias and matching behavior were too aggressive. The alias mapping that collapsed PostgreSQL into SQL was removed, while the deliberate Postgres-to-PostgreSQL alias was retained. The engine was then changed to exact normalized skill matching. A direct engine check produced a 33.33 percent skill score for the relevant opportunity, with Data Analysis matched and CSS and PostgreSQL missing. The frontend was retested afterward and no longer reported SQL as a PostgreSQL match. This test passed because the final implementation distinguishes the two technologies while still supporting explicit aliases.

FINAL STATUS: PASS

Test 04 — Kiran: Very Few Skills

The fourth test checked how the system behaves when the candidate supplies only one technical skill. Kiran's profile used Python, Bangalore, zero years of experience, and B.Tech. The purpose was to make sure the engine did not crash or behave unpredictably when the candidate has very limited skill coverage. The system successfully evaluated all opportunities and returned the top ten results. Python was shown as matched where it appeared in the opportunity requirements, while other required skills were listed as missing. Scores were lower than they would be for a broader profile, but the application remained functional. This behavior is useful because a recommendation system should still provide candidates with a starting point even when their profile is incomplete or early-career. The test also helped validate the zero-experience boundary in a realistic context. The final outcome was a successful response with ranked recommendations and no frontend failure, so the test was marked PASS.

FINAL STATUS: PASS

Test 05 — Meera: No Exact Location

This test examined the location component using a candidate whose preferred location was Mysore, while the opportunity dataset did not contain Mysore as an opportunity location. Meera's skills were Python, SQL, and Machine Learning, with two years of experience and B.Tech education. The objective was to confirm that a location mismatch reduces the score but does not eliminate every recommendation. The top result was NextGen Systems' Python Developer Intern with a score of 58.5 percent. The profile matched Machine Learning and SQL for that opportunity and satisfied the experience and education conditions. Because the location was not an exact match, the engine used the fallback location score rather than a perfect location score. The resulting score was therefore consistent with the configured weighted model. The test passed because recommendations remained available, the mismatch did not cause an exception, and the score reflected the location difference instead of treating it as an absolute rejection.

FINAL STATUS: PASS

Test 06 — Direct API POST /matches

This API-level test bypassed the frontend and sent a candidate directly to the FastAPI /matches endpoint through Swagger. The profile used Python, SQL, and Machine Learning, Bangalore, two years of experience, and B.Tech. The objective was to verify the service contract independently from browser rendering. The endpoint returned HTTP 200 and reported 30 total opportunities. It returned the top ten matches in descending order and included company, role, location, required skills, matched skills, missing skills, and match score fields. This was important because the frontend depends on exactly this JSON structure. The successful response confirmed that FastAPI could parse the request model, load the CSV data, call the matching engine, assemble result objects, sort them, and serialize them correctly. Since the API returned a complete result set without relying on frontend behavior, the test provided a clean validation of the backend layer. The final status was PASS.

FINAL STATUS: PASS

Test 07 — Missing Required Field

The seventh test deliberately omitted the education field from the API request. The purpose was to confirm that the backend rejects structurally incomplete candidate profiles instead of silently substituting a value or continuing with incomplete data. FastAPI and Pydantic returned HTTP 422. The validation response identified the missing field through a body location pointing to education and reported that the field was required. This result demonstrates that the request model is acting as a clear contract between the client and server. It also prevents incomplete education information from entering the scoring function and producing a misleading recommendation. The test was performed at the API level so that the validation behavior could be observed directly rather than being hidden behind frontend checks. Because the server correctly rejected the malformed request and provided a structured validation response, the test passed. This is an important robustness case for any API-driven recommendation application.

FINAL STATUS: PASS

Test 08 — Invalid Experience Type

The eighth test supplied an invalid data type for experience by sending the string two instead of an integer. The objective was to verify that the backend does not accept arbitrary text where a numeric experience value is required. Pydantic rejected the request with HTTP 422 and reported an `int_parsing` validation error at the body/experience location. This confirms that the request schema is not merely documenting the expected type; it actively enforces it. Rejecting invalid numeric input is important because the experience score performs arithmetic comparisons against the opportunity's minimum experience requirement. Allowing an arbitrary string into that calculation could otherwise produce inconsistent results or runtime errors. The frontend normally converts the input using `Number()`, but the API test proves that the backend remains protected even if a malformed client bypasses the browser. The test therefore passed and confirmed that invalid experience types are handled safely and predictably.

FINAL STATUS: PASS

Test 09 — Empty Skills

The ninth test used an empty skills value while providing otherwise valid information: Bangalore, one year of experience, and B.Tech. The purpose was to examine the behavior of the skill component when the candidate has not entered any skills. The API returned HTTP 200, processed all 30 opportunities, and returned ten recommendations. No skills were incorrectly marked as matched; instead, the required opportunity skills were reported as missing. The overall scores were still calculated because the scoring model also includes location, experience, and education. This is an important design behavior because an empty skill set should reduce the recommendation quality without crashing the application. The result also confirmed that the matching engine's empty-list handling returns a skill similarity of zero rather than producing a division or iteration failure. The test passed because the system remained stable, returned structured recommendations, and represented the lack of skill overlap honestly.

FINAL STATUS: PASS

Test 10 — Negative Experience

The tenth test was a defect-discovery case. A candidate request with experience equal to -1 was initially accepted by the API with HTTP 200. Although the scoring engine could technically process the value, negative years of experience are not valid for the intended profile model. The test therefore exposed a validation weakness rather than a matching error. The backend request model was strengthened by changing the experience field to use Pydantic's Field constraint with `ge=0`. The server was restarted and the same request was submitted again. The corrected response returned HTTP 422 and reported a `greater_than_equal` validation error stating that the input must be greater than or equal to zero. This was then paired with the zero-experience boundary test to ensure the new rule did not reject legitimate early-career candidates. The final outcome was PASS after the defect was fixed and retested. This test is valuable evidence of iterative engineering and input validation improvement.

FINAL STATUS: PASS

Test 11 — Zero Experience Boundary

The eleventh test verified the lower boundary introduced by the previous correction. The candidate had zero years of experience, Python as the skill, Bangalore as the location, and B.Tech education. The objective was to confirm that the new non-negative validation accepts zero while rejecting only negative values. The API returned HTTP 200 and produced a normal recommendation list. AI Labs' Python Developer Intern appeared as the top result at 45.67 percent, with Python matched and Pandas and HTML missing. The result demonstrates that zero experience is treated as a valid numeric profile rather than as malformed input. It also shows how experience requirements influence scores when the opportunity expects more experience. Together with Test 10, this case establishes a meaningful boundary: -1 is invalid, while 0 is valid. The test passed because the API accepted the boundary value, the matching engine completed normally, and the frontend could display the returned recommendations.

FINAL STATUS: PASS

Test 12 — Skill Alias Validation

The twelfth test checked the controlled skill alias system. The candidate entered ML, ReactJS, NodeJS, and Postgres rather than the exact canonical forms used by some opportunity records. The profile used Bangalore, one year of experience, and B.Tech. The API returned HTTP 200 and recognized the deliberate aliases. For example, the results showed Node.js, Machine Learning, PostgreSQL, and React as matched where appropriate. DevCore's Cloud Engineering Intern reached 69 percent with Node.js, Machine Learning, and PostgreSQL matched, while HTML and Git were missing. AI Labs' Full Stack Developer Intern also benefited from the normalized aliases. The important part of this test was not only that aliases worked, but that they did not recreate the earlier false positives between unrelated technologies. The implementation maps selected variations explicitly and then uses exact normalized equality. The test therefore passed and confirmed that alias support improves user convenience without returning to uncontrolled substring matching.

FINAL STATUS: PASS

Test 13 — Case-Insensitive Matching

The thirteenth test checked whether capitalization affects recommendation results. The candidate entered PYTHON, sql, and MACHINE LEARNING using inconsistent capitalization, with Bangalore, two years of experience, and B.Tech. The expected behavior was that these values should normalize to the same canonical forms as lowercase input. The returned recommendations were consistent with the earlier clean-case baseline. NextGen Systems' Python Developer Intern scored 58.5 percent and correctly reported Machine Learning and SQL as matched. CloudWorks' Full Stack Developer Intern scored 57.5 percent and reported SQL and Python as matched. No additional false matches were introduced by the capitalization changes. This behavior comes from the normalization step, which strips surrounding whitespace, converts skills to lowercase, and then applies aliases. The test passed because the system produced the same meaningful skill relationships and ranking pattern regardless of capitalization. This is important for real user input because candidates rarely enter skills with perfectly standardized capitalization.

FINAL STATUS: PASS

Test 14 — Extra Spaces and Formatting

The fourteenth test targeted common formatting noise. The candidate entered the skills string with extra leading and trailing spaces around Python, SQL, and Machine Learning, while keeping Bangalore, two years of experience, and B.Tech. The objective was to confirm that formatting differences do not create artificial missing skills or change the recommendation order. The API returned a successful response and produced the same recommendation pattern and scores as the case-normalization test. This is explained by the clean_skills function, which splits the input on commas or semicolons, strips surrounding whitespace, converts values to lowercase, applies aliases, and removes duplicates. The test therefore validated the preprocessing layer independently of the scoring weights. It also confirmed that the frontend can accept human-entered formatting rather than requiring a rigid syntax. The result was PASS because the additional spaces were effectively ignored and the engine still recognized the intended skills correctly.

FINAL STATUS: PASS

Test 15 — Unknown Skills

The fifteenth test used skills that were not present in the opportunity skill vocabulary: Kubernetes, Rust, and Solidity. The profile used Bangalore, one year of experience, and B.Tech. The purpose was to ensure that the system does not invent matches simply because a candidate has supplied unfamiliar text. The API returned HTTP 200 and produced normal recommendations. The top WebWorks AI Engineering Intern result scored 34 percent, with no matched skills and all of its required skills listed as missing. This is the expected behavior: unknown candidate skills should not be treated as equivalent to unrelated known skills. The remaining score contribution comes from other factors such as location, experience, and education, so the final score is not necessarily zero. The test passed because the engine remained stable, returned no false skill overlap, and still provided a useful ranked response. This case is especially relevant when the dataset vocabulary is smaller than the real-world skill space.

FINAL STATUS: PASS

Test 16 — Location and Education Variation

This test deliberately varied both location and education to verify that the two contextual components affect the final recommendation. The candidate profile used Python, SQL, and Machine Learning, with Mysore as the location, two years of experience, and MCA education. The screenshot from the run shows a successful recommendation list, with CloudWorks' Full Stack Developer Intern appearing at 53.5 percent. That opportunity had SQL and Python matched, CSS and AWS missing, and a Hyderabad location. The candidate therefore did not receive the perfect location component. The education was also not an exact match for the listed requirement, so the education component was reduced rather than treated as a full match. Experience remained fully satisfied. The resulting score is consistent with the weighted scoring model. The test passed because the system handled the two variations without error and produced a score that reflected the differences rather than rejecting the candidate outright.

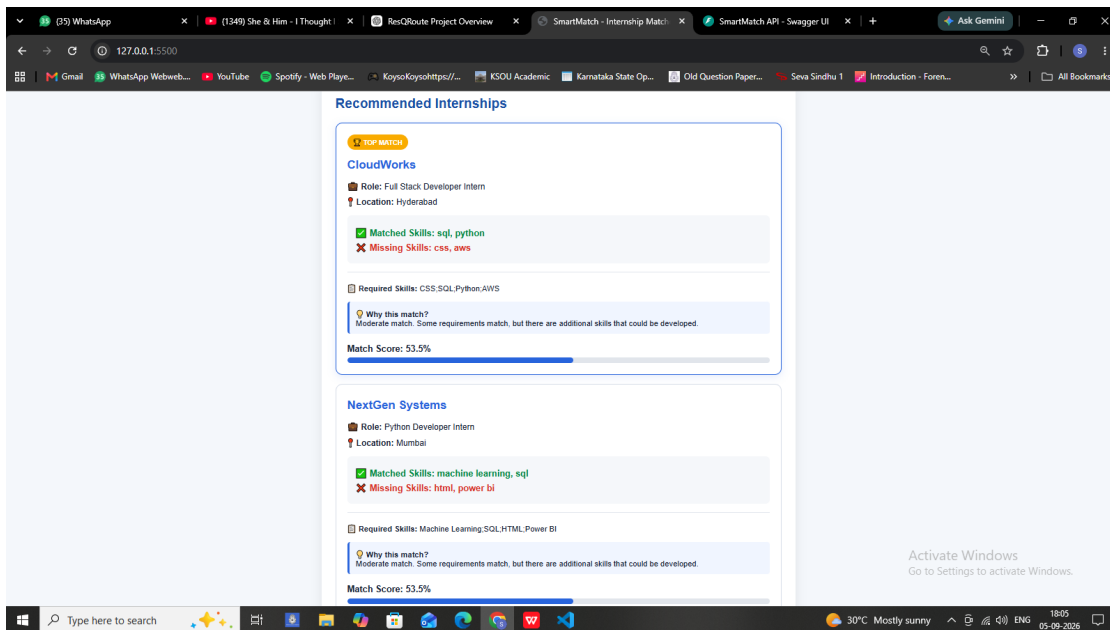


Figure 1. Test 16 — location and education variation results.

FINAL STATUS: PASS

Test 17 — Ranking Validation

The seventeenth test checked the ordering guarantee of the recommendation list. The candidate used Python, SQL, and Machine Learning, Bangalore, two years of experience, and B.Tech. The returned scores were 58.50, 57.50, 55.67, 50.25, 47.67, 44.83, 43.00, 42.67, 42.67, and 41.25 percent. Every value was equal to or lower than the one above it, and the two 42.67 percent results formed a valid tie. This directly verifies the backend sorting operation, which orders results using match_score in reverse order before slicing the first ten records. The test also confirms that the frontend preserves the backend order instead of applying its own ranking. The first record was correctly treated as the top match. Because no descending-order violation occurred, the ranking requirement passed. This test is important because a recommendation engine is only useful if the displayed shortlist actually reflects the score it calculated.

FINAL STATUS: PASS

Test 18 — Final Frontend End-to-End Test

The final test validated the complete user journey rather than one isolated component. A candidate profile was entered through the browser and submitted using the SmartMatch interface. The screenshot shows the resulting recommendation cards rendered successfully. The page displayed company names, roles, locations, matched skills, missing skills, required skills, explanation text, match scores, and visual score bars. The first recommendation was highlighted with the TOP MATCH badge. The successful display confirms that the browser collected the profile, constructed the JSON request, reached the FastAPI service, received a valid response, and passed the returned data into the result-rendering function. It also confirms that the backend response fields introduced during earlier fixes are now visible in the final UI. No connection or rendering error was observed. This test therefore represents the final integration checkpoint for the prototype. It passed and completed the campaign with an overall result of 18 successful tests out of 18 conducted.

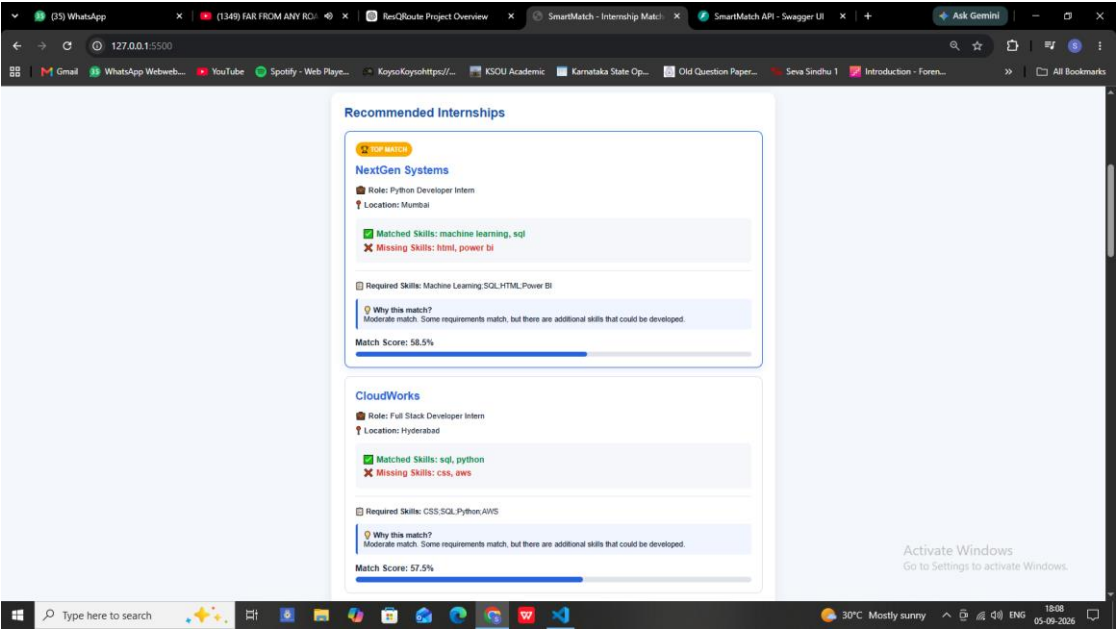


Figure 2. Test 18 — final frontend end-to-end recommendation results.

FINAL STATUS: PASS

22. Final Testing Summary

Test	Focus	Status
01	Rahul: Normal Multi-Skill Profile	PASS
02	Arjun: Frontend Profile	PASS
03	Priya: SQL/PostgreSQL Validation	PASS
04	Kiran: Very Few Skills	PASS
05	Meera: No Exact Location	PASS
06	Direct API POST /matches	PASS
07	Missing Required Field	PASS
08	Invalid Experience Type	PASS
09	Empty Skills	PASS
10	Negative Experience	PASS
11	Zero Experience Boundary	PASS

12	Skill Alias Validation	PASS
13	Case-Insensitive Matching	PASS
14	Extra Spaces and Formatting	PASS
15	Unknown Skills	PASS
16	Location and Education Variation	PASS
17	Ranking Validation	PASS
18	Final Frontend End-to-End Test	PASS

Overall result: 18 / 18 tests passed (100%).

The validation campaign covered functional recommendations, API behavior, input validation, skill normalization, alias handling, edge cases, contextual scoring, ranking order, and the complete browser-to-API-to-matching-engine flow.

23. References and Source Basis

- backend/main.py — FastAPI service, request model, CORS configuration, and /matches endpoint.
- ml-engine/matching_engine.py — skill normalization, aliases, scoring functions, explanations, and batch matching.
- frontend/index.html — candidate form, API request logic, result cards, score display, and frontend validation.
- data/interns.csv — synthetic intern dataset used by the batch matching engine.
- data/opportunities.csv — synthetic internship opportunity dataset used by the API.
- data/matches.csv — generated pairwise match output for stored intern and opportunity records.
- data/generate_interns.py and data/generate_opportunities.py — deterministic synthetic data generation using seed 42.
- Completed manual/API validation record for Tests 1–18 and screenshots supplied during testing.

Output

Find Your Best Internship

SmartMatch AI-powered internship recommendation system

Name

Rahul

Skills

Python, SQL, Machine Learning

Location

Bangalore

Experience (Years)

2

Education

B.Tech

Find Matching Internships

Recommended Internships

TOP MATCH

NextGen Systems

Role: Python Developer Intern

Location: Mumbai

✓ Matched Skills: machine learning, sql

✗ Missing Skills: html, power bi

Required Skills: Machine Learning;SQL;HTML;Power BI

Why this match?

Moderate match. Some requirements match, but there are additional skills that could be developed.

Match Score: 58.5%

CloudWorks

Role: Full Stack Developer Intern

Location: Hyderabad

✓ Matched Skills: sql, python

✗ Missing Skills: css, aws

Required Skills: CSS;SQL;Python;AWS

Why this match?

Moderate match. Some requirements match, but there are additional skills that could be developed.

Match Score: 57.5%

WebWorks

 Role: AI Engineering Intern

 Location: Bangalore

 **Matched Skills:** react

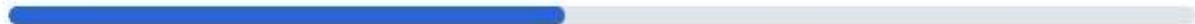
 **Missing Skills:** css, power bi, mongodb, html

 **Required Skills:** React;CSS;Power BI;MongoDB;HTML

 **Why this match?**

Lower match. Consider improving the missing skills before applying.

Match Score: 47%



WebWorks

 Role: Machine Learning Intern

 Location: Kochi

 **Matched Skills:** react, machine learning

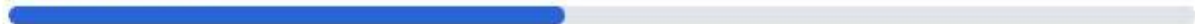
 **Missing Skills:** css, power bi, pandas

 **Required Skills:** React;CSS;Power BI;Machine Learning;Pandas

 **Why this match?**

Lower match. Consider improving the missing skills before applying.

Match Score: 47%



ByteLabs

 Role: Web Development Intern

 Location: Delhi

✅ **Matched Skills:** react, sql

❌ **Missing Skills:** aws, pandas

 **Required Skills:** React,AWS;SQL;Pandas

 **Why this match?**

Moderate match. Some requirements match, but there are additional skills that could be developed.

Match Score: 57.5%

AI Labs

 Role: Python Developer Intern

 Location: Bangalore

✅ **Matched Skills:** python

❌ **Missing Skills:** pandas, html

 **Required Skills:** Python;Pandas;HTML

 **Why this match?**

Moderate match. Some requirements match, but there are additional skills that could be developed.

Match Score: 55.67%